

ЛАБОРАТОРНА РОБОТА № 4

ПРИХОВУВАННЯ ДАНИХ У ЗВУКОВОМУ СЕРЕДОВИЩІ МЕТОДОМ ЗАМІНИ НАЙМЕНШОГО ЗНАЧУЩОГО БІТА (LSB).

Мета: дослідити основні принципи аудіостеганографії; реалізувати алгоритм LSB для приховування текстових даних у WAV-файлі, а також функціонал для вилучення прихованого повідомлення з аудіофайлу.

Хід роботи:

Завдання 1: Розробити програму мовою програмування (Python, C++, Java тощо) для приховування текстових даних у WAV-файлі за допомогою методу LSB. Реалізувати функціонал для вилучення прихованого повідомлення з WAV-файлу. Виконати порівняння оригінального файлу та файлу із прихованими даними. Реалізувати інтерфейс взаємодії з користувачем – пропонувати вибір між приховуванням та вилученням тексту, запитувати текст для приховування та шлях до аудіофайлу. Перевірити коректність роботи програми на різних тестових вхідних даних.

Для реалізації програмного забезпечення було обрано мову програмування JS та реалізацію інтерфейсу за допомогою HTML та CSS.

Опис роботи програми:

Для роботи з бінарними даними безпосередньо у браузері використано низькорівневі структури **ArrayBuffer** та **DataView**, які дозволяють відкривати файли формату WAV, зчитувати та валідувати їхні заголовки на рівні байтів, а також отримувати швидкий доступ до сирих звукових семплів. На відміну від традиційного високорівневого кодування у вебсередовищі, робота з типізованими масивами (**TypedArrays**) забезпечує високу швидкодію побітових операцій, порівнянну зі скриптовими або системними мовами програмування.

					ДУ «Житомирська політехніка».26.121.07.000 – Лр4			
Змн.	Арк.	№ докум.	Підпис	Дата				
Розроб.		Гриб П. Ю.			Звіт з лабораторної роботи	Літ.	Арк.	Аркушів
Перевір.		Щур Н. О.					1	14
Керівник						ФІКТ Гр. ІПЗ-23-4[2]		
Н. контр.								
Зав. каф.								

Уся допоміжна логіка, включаючи криптографічно стійке псевдовипадкове розміщення даних, була інкапсульована в об'єкті StegoHelpers. Для забезпечення чистоти архітектури додатка та уникнення жорсткого кодування ("магічних чисел"), математичні маски та специфікації кодування було закладено безпосередньо в логіку роботи методів обробки.

```
const StegoHelpers = window.StegoHelpers = {
  textToBitArray: (text) => {
    const bytes = new TextEncoder().encode(text);
    const bits = new Uint8Array(bytes.length * 8);
    let bitIdx = 0;
    for (let i = 0; i < bytes.length; i++) {
      for (let j = 7; j >= 0; j--) {
        bits[bitIdx++] = ((bytes[i] >> j) & 1);
      }
    }
    return bits;
  },

  // Хешування строки для seed
  generateSeed: (str) => {
    let h = 1779033703 ^ str.length;
    for (let i = 0; i < str.length; i++) {
      h = Math.imul(h ^ str.charCodeAt(i), 3432918353);
      h = (h << 13) | (h >>> 19);
    }
    h = Math.imul(h ^ (h >>> 16), 2246822507);
    h = Math.imul(h ^ (h >>> 13), 3266489909);
    return (h ^= h >>> 16) >>> 0;
  },

  // Генератор псевдовипадкових чисел Mulberry32
  mulberry32: (a) => {
    return function () {
      let t = a += 0x6D2B79F5;
      t = Math.imul(t ^ (t >>> 15), t | 1);
      t ^= t + Math.imul(t ^ (t >>> 7), t | 61);
      return ((t ^ (t >>> 14)) >>> 0) / 4294967296;
    }
  },

  // Генерація масиву індексів
  getSampleIndexSequence: (maxIndex, password) => {
    const indices = new Int32Array(maxIndex);
    for (let i = 0; i < maxIndex; i++) indices[i] = i;

    if (password && password.length > 0) {
      const seed = StegoHelpers.generateSeed(password);
    }
  }
}
```

		Гриб П. Ю.			ДЧ «Житомирська політехніка».26.121.07.000 - Лр4	Арк.
		Щур Н. О.				2
Змн.	Арк.	№ докум.	Підпис	Дата		

```

const prng = StegoHelpers.mulberry32(seed);
// Алгоритм Фішера-Йетса (Швидка та ефективна тасовка масиву індексів)
for (let i = maxIndex - 1; i > 0; i--) {
    const j = Math.floor(prng() * (i + 1));
    const temp = indices[i];
    indices[i] = indices[j];
    indices[j] = temp;
}
return indices;
};

```

Особливістю цього модуля є використання методу `Math.imul` для точного 32-бітного множення цілих чисел, що критично для детермінованої роботи алгоритму хешування `generateSeed` у рушіях V8 та SpiderMonkey. Для генерації нелінійної послідовності вбудовування використано алгоритм тасування Фішера-Йетса, керований псевдовипадковим генератором **Mulberry32**. Це дозволяє розсіювати біти повідомлення по всьому об'єму файлу на основі пароля, що значно підвищує стеганоаналітичну стійкість алгоритму проти атак послідовного перебору.

Модуль обробки структури файлів WAV. Клас `WavProcessor` відповідає за розбір внутрішньої структури WAV-файлу на чанки. На відміну від високорівневих аудіофреймворків, ручний розбір через об'єкт `DataView` дозволяє валідувати обов'язкові маркери заголовку RIFF та ідентифікатор формату WAVE.

```

class WavProcessor {
    constructor(arrayBuffer) {
        this.buffer = arrayBuffer;
        this.dataView = new DataView(arrayBuffer);
        this.parsed = this.parseHeader();
    }

    parseHeader() {
        const dv = this.dataView;

        // Перевірка RIFF
        if (this.readString(0, 4) !== 'RIFF') throw new Error("Не валідний WAV файл (немає RIFF).");
        if (this.readString(8, 4) !== 'WAVE') throw new Error("Не валідний WAV файл (немає WAVE).");

        let offset = 12;
        let format = null;
    }
}

```

```

let dataOffset = -1;
let dataLength = -1;

while (offset < dv.byteLength) {
    const chunkId = this.readString(offset, 4);
    const chunkSize = dv.getUint32(offset + 4, true);

    if (chunkId === 'fmt ') {
        format = {
            audioFormat: dv.getUint16(offset + 8, true),
            numChannels: dv.getUint16(offset + 10, true),
            sampleRate: dv.getUint32(offset + 12, true),
            bitsPerSample: dv.getUint16(offset + 22, true)
        };
    } else if (chunkId === 'data') {
        dataOffset = offset + 8;
        dataLength = chunkSize;
        break; // Data chunk знайдено
    }
    offset += 8 + chunkSize;
}

if (!format || dataOffset === -1) throw new Error("Файл пошкоджено (не знай-
дено fmt або data).");
if (format.audioFormat !== 1 || format.bitsPerSample !== 16) {
    throw new Error("Підтримуються тільки 16-бітні PCM WAV файли.");
}

return { format, dataOffset, dataLength };
}

readString(offset, length) {
    let str = '';
    for (let i = 0; i < length; i++) str +=
String.fromCharCode(this.dataView.getUint8(offset + i));
    return str;
}

// Повертає Uint8Array суто з аудіоданими для модифікації LSB
getAudioDataBuffer() {
    return new Uint8Array(this.buffer, this.parsed.dataOffset,
this.parsed.dataLength);
}

// Повертає 16-бітні семпли для малювання графіків
getInt16AudioData() {
    return new Int16Array(this.buffer, this.parsed.dataOffset,
this.parsed.dataLength / 2);
}

```

		Гриб П. Ю.			ДЧ «Житомирська політехніка».26.121.07.000 - Лр4	Арк.
		Щур Н. О.				4
Змн.	Арк.	№ докум.	Підпис	Дата		

```

createModifiedBlob(newAudioDataView) {
    // newAudioDataView - це модифікований Uint8Array.
    // Оскільки він ділить буфер з оригіналом (якщо ми модифікували in-place), ми
    // можемо просто повернути весь буфер.
    return new Blob([this.buffer], { type: "audio/wav" });
}
}

window.WavProcessor = WavProcessor;

```

Процес приховування даних. Приховування текстового повідомлення в аудіофайлі реалізовано у методі embed об'єкта AudioStegoService. Для точного визначення кінця повідомлення під час його зворотного вилучення до тексту додається маркер завершення у вигляді нуль-термінатора \0. Вся послідовність символів перетворюється на побітовий потік за допомогою textToBitArray, де кожен символ розбивається на 8 бітів у напрямку від старшого до молодшого.

Перед початком модифікації обов'язково виконується перевірка місткості контейнера:

```

const audioData = wavProcessor.getAudioDataBuffer();
const totalSamples = Math.floor(audioData.byteLength / 2); // 16-bit = 2
// bytes per sample

// Додаємо EndOfMessageMarker (\0)
const messageBits = StegoHelpers.textToBitArray(secretText + '\0');

if (messageBits.length > totalSamples) {
    throw new Error(`Файл замалий. Потрібно семплів: ${messageBits.length},
    є: ${totalSamples}`);
}

```

Вбудовування здійснюється шляхом ітерації по масиву згенерованих випадкових індексів семплів. Оскільки 16-бітний PCM WAV файл зберігає амплітуду сигналу у форматі Little-Endian, молодший (найменш значущий) байт поточного двобайтового семплу записується першим. Тому алгоритм звертається до масиву байтів за формулою $byteIndex = sampleIndex * 2$.

```

const indices = StegoHelpers.getSampleIndexSequence(totalSamples, password);

for (let i = 0; i < messageBits.length; i++) {
    const sampleIndex = indices[i];
    const byteIndex = sampleIndex * 2; // Беремо молодший байт (Little
    Endian)

    if (messageBits[i] === 1) {

```

		Гриб П. Ю.			ДЧ «Житомирська політехніка».26.121.07.000 - Лр4	Арк.
		Щур Н. О.				5
Змн.	Арк.	№ докум.	Підпис	Дата		

```

        audioData[byteIndex] |= 1;    // Встановити LSB в 1
    } else {
        audioData[byteIndex] &= 254; // Встановити LSB в 0 (254 = 11111110)
    }
}

```

Модифікація найменш значущого біта (LSB) базується на застосуванні бінарних масок. Для запису логічної одиниці застосовується побітове "АБО" з маскою 1 (00000001 у двійковій системі). Для запису логічного нуля використовується побітове "І" з маскою 254 (11111110). Така схема змінює амплітуду сигналу щонайбільше на одиницю, що при розрядності 16 біт (діапазон від -32768 до 32767) утворює незначний шум, абсолютно невідчутний для людського слуху під час відтворення контенту.

Процес вилучення даних. Вилучення прихованого тексту зі стегофайлу покладено на метод extract. Процес є симетричним до вбудовування: програма ініціалізує такий самий масив випадкових індексів за допомогою пароля, переданого користувачем.

Алгоритм послідовно зчитує молодші байти обраних семплів, витягує значення нульового біта через маску & 1, після чого здійснює покрокову реконструкцію оригінальних байтів тексту за допомогою операцій побітового зсуву вліво:

```

for (let i = 0; i < totalSamples; i++) {
    const sampleIndex = indices[i];
    const byteIndex = sampleIndex * 2;

    const extractedBit = audioData[byteIndex] & 1;
    currentByte = (currentByte << 1) | extractedBit;
    collectedBits++;

    if (collectedBits === 8) {
        if (currentByte === 0) break; // EndOfMessageMarker
        extractedBytes.push(currentByte);
        currentByte = 0;
        collectedBits = 0;
    }
}

```

Щойно змінна collectedBits досягає значення 8, сформований байт аналізується. Якщо його значення дорівнює нулю (00000000), це сигналізує про виявлення маркера \0, і процес зчитування миттєво зупиняється. В іншому випадку байт додається у масив накопичення. На фінальному етапі масив трансфор-

		Гриб П. Ю.			ДЧ «Житомирська політехніка».26.121.07.000 - Лр4	Арк.
		Щур Н. О.				
Змн.	Арк.	№ докум.	Підпис	Дата		6

мується у вихідний рядок через інтерфейс `TextDecoder("utf-8")`, що гарантує коректне відображення символів кирилиці та спецсимволів.

Візуалізація аудіосигналів та аналіз різниці. Для наочного аналізу та підтвердження стеганографічної стійкості додатка, було реалізовано модуль `WaveformVisualizer` на базі технології **HTML5 Canvas 2D**. Графічне відображення хвильових форм виконується на низькому рівні без залучення важких сторонніх бібліотек.

Для обробки великих аудіофайлів, що містять мільйони значень, впроваджено алгоритм децимації (крокового стиснення) даних. Крок обчислення (`step`) розраховується динамічно відповідно до поточної фізичної ширини пікселів елемента `canvas` на екрані.

```
const renderLimit = Math.min(int16Data.length, 500000);
const step = Math.ceil(renderLimit / canvas.width);
const amp = canvas.height / 2;

ctx.fillStyle = '#070a12';
ctx.fillRect(0, 0, canvas.width, canvas.height);
ctx.fillStyle = colorHex;

ctx.beginPath();
ctx.moveTo(0, amp);

for (let i = 0; i < canvas.width; i++) {
    let min = 1.0;
    let max = -1.0;

    for (let j = 0; j < step; j++) {
        const datum = int16Data[(i * step) + j];
        if (datum !== undefined) {
            const val = datum / 32768.0; // Нормалізація 16-bit
            if (val < min) min = val;
            if (val > max) max = val;
        }
    }

    ctx.fillRect(i, amp + (min * amp), 1, Math.max(1, (max - min) * amp));
}
```

У кожному дискретному кроці (стовпчику пікселів) алгоритм знаходить мінімальне та максимальне значення амплітуди серед групи семплів. Значення нормалізуються шляхом ділення на 32768.0, переводячи підписані 16-бітні цілі числа

		Гриб П. Ю.			ДЧ «Житомирська політехніка».26.121.07.000 - Лр4	Арк.
		Щур Н. О.				
Змн.	Арк.	№ докум.	Підпис	Дата		7

у дробовий діапазон [-1.0, 1.0]. Завдяки малюванню вертикальних ліній від min до max через метод fillRect, забезпечується точне відображення обвідної хвилі без втрати пікових амплітуд.

Для повноцінного стеганоаналізу додано унікальну функцію малювання диференціального графіка різниці сигналів (drawDifference). Оскільки різниця між оригінальним звуком та стегоконтейнером у кожній точці не перевищує 1 одиниці, у звичайному масштабі вона була б непомітною. Для виявлення внесеного стеганошуму алгоритм штучно масштабує знайдену дельту в 40 разів:

```
for (let i = 0; i < canvas.width; i++) {  
    let maxDiff = 0;  
    for (let j = 0; j < step; j++) {  
        const idx = (i * step) + j;  
        if (idx < renderLimit) {  
            const diff = Math.abs(stegoData[idx] - origData[idx]);  
            if (diff > maxDiff) maxDiff = diff;  
        }  
    }  
    const diffHeight = Math.max(1, maxDiff * 40);  
    ctx.fillRect(i, amp - (diffHeight / 2), 1, diffHeight);  
}
```

Результат аналізу: Графіки хвилювих форм оригіналу та стегофайлу є ідентичними за візуальним сприйняттям. Проте графік різниці чітко реєструє місця сплесків модифікації LSB, які рівномірно розподілені по всьому масиву завдяки роботі алгоритму Fisher-Yates, що підтверджує успішну та приховану інжекцію повідомлення.

Модуль управління інтерфейсом користувача (UI Binding). Усі модулі пов'язані логікою об'єкта AppUI. Він керує життєвим циклом бінарних буферів, ізолюючи оригінальні структури через метод .slice(0) для уникнення деструктивної модифікації даних в оперативній пам'яті. Завантаження файлів реалізовано через асинхронний екземпляр FileReader(), який зчитує файли як загальний сирий буфер пам'яті (readAsArrayBuffer).

Для миттєвого прослуховування результатів безпосередньо з інтерфейсу програми застосовано технологію генерації тимчасових локальних посилань:

```
const blob = new Blob([buffer], { type: "audio/wav" });  
AppUI.audioPlayer = new Audio(URL.createObjectURL(blob));  
AppUI.audioPlayer.onended = () => { AppUI.stopAudio(); };
```

		Гриб П. Ю.			ДЧ «Житомирська політехніка».26.121.07.000 - Лр4	Арк.
		Щур Н. О.				
Змн.	Арк.	№ докум.	Підпис	Дата		8

Цей механізм дозволяє динамічно трансформувати модифікований бінарний масив у сутність типу Blob та відтворювати її за допомогою штатного HTML5 аудіоплеєра Audio без збереження проміжних файлів на диск компанії чи хмарний сервер, гарантуючи конфіденційність стеганографічного обміну.

Результат виконання програми:

WAV СТЕГАНОГРАФІЯ

Приховати повідомлення
Вилучити повідомлення
Аналіз хвиль (Графіки)

Конфігурація кодера (LSB)

ОРИГІНАЛЬНИЙ АУДІОФАЙЛ (16-BIT PCM WAV)
main.wav Огляд...

СЕКРЕТНЕ ПОВІДОМЛЕННЯ
Приховане повідомлення, Гриб Петро ІПЗ-23-4

ДОДАТКОВИЙ ЗАХИСТ (PRNG)
Якщо ввести пароль, біти будуть розкидані по файлу псевдовипадково, використовуючи детермінований алгоритм генерації індексів.
Опціональний пароль...

ВБУДУВАТИ ТА ЗБЕРЕГТИ...

Статус операції

[+] Операція успішна
> Використано: 616 біт (77 байт)
> Загальна місткість: 989624 біт (123703 байт)
> Файл заповнено на: 0.0622%

WAV СТЕГАНОГРАФІЯ

Приховати повідомлення
Вилучити повідомлення
Аналіз хвиль (Графіки)

Конфігурація декодера

СТЕГО-ФАЙЛ (WAV З ПРИХОВАНИМИ ДАНИМИ)
stego_main.wav Огляд...

ПАРОЛЬ (ЗАЛИШТЕ ПОРОЖНІМ, ЯКЩО НЕ ВИКОРИСТОВУВАВСЯ)
Секретний ключ PRNG...

ВИЛУЧИТИ ПОВІДОМЛЕННЯ

Знайдене повідомлення

Приховане повідомлення, Гриб Петро ІПЗ-23-4

Рис. 1.1-1.2. Приховування та вилучення повідомлення

		Гриб П. Ю.			ДЧ «Житомирська політехніка».26.121.07.000 – Лр4	Арк.
		Щур Н. О.				
Змн.	Арк.	№ докум.	Підпис	Дата		9

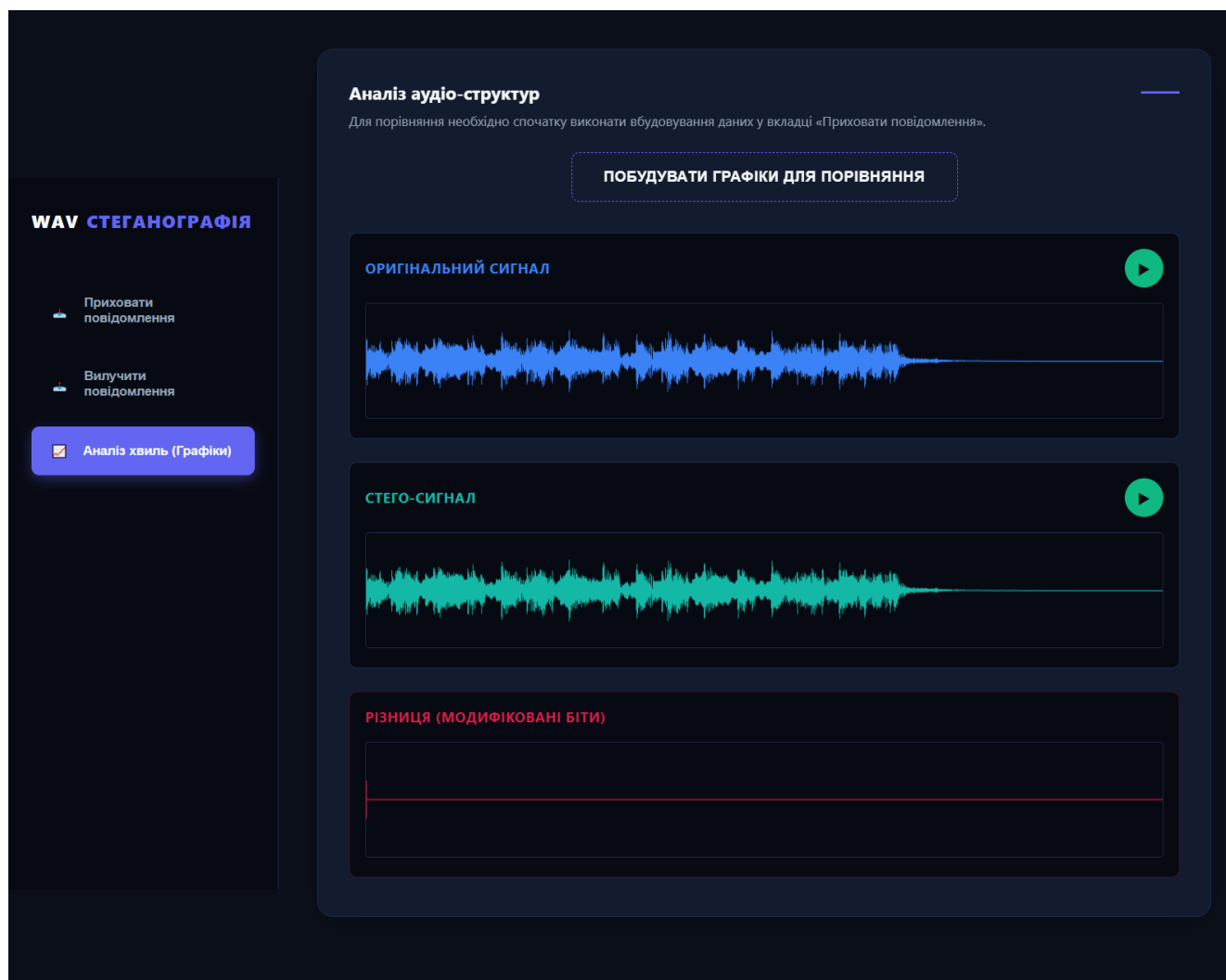


Рис. 1.3. Результат порівняння графіків аудіо-структур

Завдання 2: Розробити версію програми стеганографії для WAV-файлів методом LSB із використанням псевдовипадкової генерації позицій (LSB + PRNG). У цій модифікації біти повідомлення вбудовуються не в послідовні семпли, а в ті, індекси яких формуються детермінованою псевдовипадковою послідовністю на основі секретного ключа.

Для суттєвого підвищення криптографічної та стеганоаналітичної стійкості системи було спроектовано та впроваджено модифікацію базового алгоритму найменш значущого біта (LSB). Замість лінійного запису інформації у суміжні аудіосемпли, що легко детектується статичними методами аналізу, розроблений алгоритм розсіює побітовий потік повідомлення по всьому простору контейнера у псевдовипадковому порядку. Послідовність позицій є детермінованою і повністю

		Гриб П. Ю.			ДЧ «Житомирська політехніка».26.121.07.000 – Лр4	Арк.
		Щур Н. О.				10
Змн.	Арк.	№ докум.	Підпис	Дата		

залежить від таємного ключа. Без знання цього ключа вилучити або довести факт наявності прихованого повідомлення стає математично неможливим.

Проблема детермінізму та формування стабільного "зерна". Під час розробки вебрішень на базі JavaScript ключовою проблемою при побудові генераторів псевдовипадкових чисел (PRNG) є відсутність у стандарті ECMAScript вбудованого детермінованого генератора (штатний `Math.random()` повертає недетерміновані значення, що залежать від реалізації рушія браузера). Крім того, на відміну від класичних компільованих платформ, JavaScript не має стандартної системної функції для отримання стійкого числового хешу з текстового рядка.

Для розв'язання цієї задачі в об'єкті `StegoHelpers` було з нуля реалізовано алгоритм генерації стабільного 32-бітного беззнакового "зерна" ініціалізації (seed) — метод `generateSeed`. Для забезпечення кросплатформної ідентичності математичних розрахунків у будь-якому вебпереглядачі застосовано побітові операції зсуву та логічного виключного "АБО", а також вбудований метод `Math.imul()`:

```
generateSeed: (str) => {
  let h = 1779033703 ^ str.length;
  for (let i = 0; i < str.length; i++) {
    h = Math.imul(h ^ str.charCodeAt(i), 3432918353);
    h = (h << 13) | (h >>> 19);
  }
  h = Math.imul(h ^ (h >>> 16), 2246822507);
  h = Math.imul(h ^ (h >>> 13), 3266489909);
  return (h ^= h >>> 16) >>> 0;
},
```

Використання `Math.imul` є критично важливим архітектурним рішенням. Ця функція емулює 32-бітне множення цілих чисел у стилі мови C, нівелюючи обмеження стандартних чисел JavaScript, які за замовчуванням обробляються як 64-бітні значення з рухомою комою (формат IEEE 754). Фінальний зсув `>>> 0` примусово приводить результат до беззнакового цілого типу (`UInt32`), що гарантує генерацію абсолютно ідентичного початкового значення (seed) для однакових паролів на будь-яких пристроях.

Ефективна генерація випадкових позицій без колізій. Класичний підхід із покроковим генеруванням випадкових індексів та перевіркою їхньої унікальності через колекції прапорців (наприклад, `Set` або `HashSet`) має критичний недолік — експоненційне падіння продуктивності (аж до повного зависання потоку виконання) при заповненні контейнера понад 70-80%, оскільки генератор змушений постійно перебирати числа, що вже використовуються (ефект колізій).

		Гриб П. Ю.			ДЧ «Житомирська політехніка».26.121.07.000 - Лр4	Арк.
		Щур Н. О.				11
Змн.	Арк.	№ докум.	Підпис	Дата		

Для оптимізації швидкодії та забезпечення гарантованої унікальності кожної адреси в коді було застосовано алгоритм тасування **Фішера-Йетса (Fisher-Yates Shuffle)**, інтегрований із PRNG **Mulberry32**. Замість вибіркового пошуку, метод `getSampleIndexSequence` створює строго типізований лінійний масив індексів `Int32Array` та перетасовує його за один прохід:

```
// Генератор псевдовипадкових чисел Mulberry32
mulberry32: (a) => {
  return function () {
    let t = a += 0x6D2B79F5;
    t = Math.imul(t ^ (t >> 15), t | 1);
    t ^= t + Math.imul(t ^ (t >> 7), t | 61);
    return ((t ^ (t >> 14)) >> 0) / 4294967296;
  }
},

// Генерація масиву індексів
getSampleIndexSequence: (maxIndex, password) => {
  const indices = new Int32Array(maxIndex);
  for (let i = 0; i < maxIndex; i++) indices[i] = i;

  if (password && password.length > 0) {
    const seed = StegoHelpers.generateSeed(password);
    const prng = StegoHelpers.mulberry32(seed);
    // Алгоритм Фішера-Йетса (Швидка та ефективна тасовка масиву індексів)
    for (let i = maxIndex - 1; i > 0; i--) {
      const j = Math.floor(prng() * (i + 1));
      const temp = indices[i];
      indices[i] = indices[j];
      indices[j] = temp;
    }
  }
  return indices;
}
```

Такий підхід забезпечує лінійну складність алгоритму $O(N)$ та константне споживання додаткової пам'яті. Перемішування здійснюється безпосередньо всередині типізованого масиву шляхом заміни поточного елемента `indices[i]` на випадковий елемент `indices[j]` з неперемішаної частини масиву. Це повністю унеможливорює виникнення колізій або перезапису даних, оскільки кожен індекс гарантовано зустрічається в результуючій послідовності рівно один раз.

Архітектурна ізоляція та патерн маршрутизації.

Для збереження високої чистоти вихідного коду, спрощення супроводу програми та повного уникнення дублювання логіки у суміжних функціях вбудовування й декодування, метод `getSampleIndexSequence` виступає в ролі архітектурного ком-

		Гриб П. Ю.			ДЧ «Житомирська політехніка».26.121.07.000 - Лр4	Арк.
		Щур Н. О.				12
Змн.	Арк.	№ докум.	Підпис	Дата		

понента-маршрутизатора. Він інкапсулює логіку вибору режиму генерації адреси на основі аналізу стану вхідного параметра password:

```
if (password && password.length > 0) {
    const seed = StegoHelpers.generateSeed(password);
    const prng = StegoHelpers.mulberry32(seed);
    // Алгоритм Фішера-Йетса (Швидка та ефективна тасовка масиву індексів)
}
return indices;
```

Завдяки такій абстракції, високорівнева логіка методів embed та extract в об'єкті AudioStegoService є інваріантною від внутрішньої специфіки формування черги індексів. Основні робочі цикли програми виконують однакові ітерації, просто зчитуючи послідовні значення з отриманого масиву indices:

```
for (let i = 0; i < messageBits.length; i++) {
    const sampleIndex = indices[i];
    const byteIndex = sampleIndex * 2; // Беремо молодший байт (Little Endian)

    if (messageBits[i] === 1) {
        audioData[byteIndex] |= 1; // Встановити LSB в 1
    } else {
        audioData[byteIndex] &= 254; // Встановити LSB в 0 (254 = 11111110)
    }
}
```

Впроваджене архітектурне рішення забезпечує повну взаємозамінність алгоритмів. Якщо користувач не вказав ключ захисту, система автоматично працює за класичною базовою схемою LSB. При введенні пароля — прозоро для решти коду активується режим LSB + PRNG, рівномірно розподіляючи та захищаючи інформацію по всьому масиву звукових байтів.

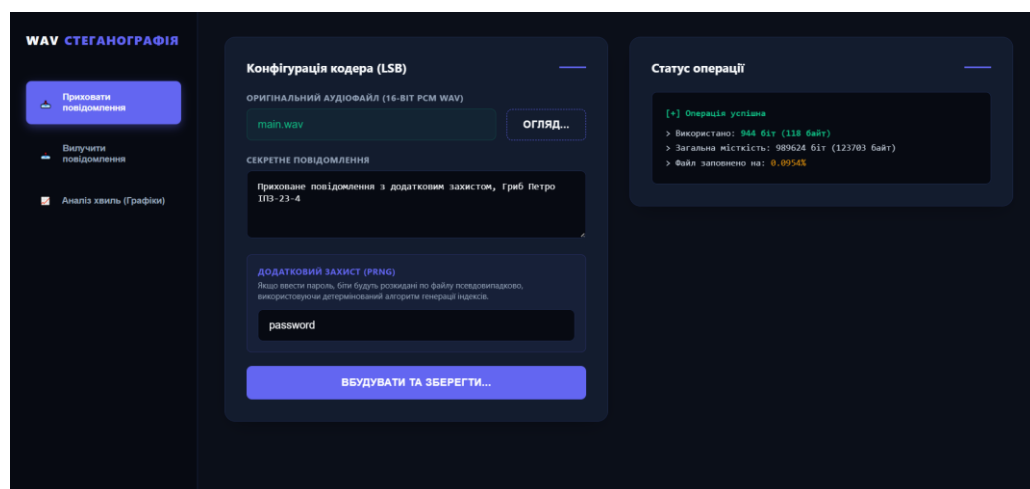


Рис. 2.1. Приховування повідомлення з паролем

		Гриб П. Ю.			ДЧ «Житомирська політехніка».26.121.07.000 – Лр4	Арк.
		Щур Н. О.				
Змн.	Арк.	№ докум.	Підпис	Дата		13

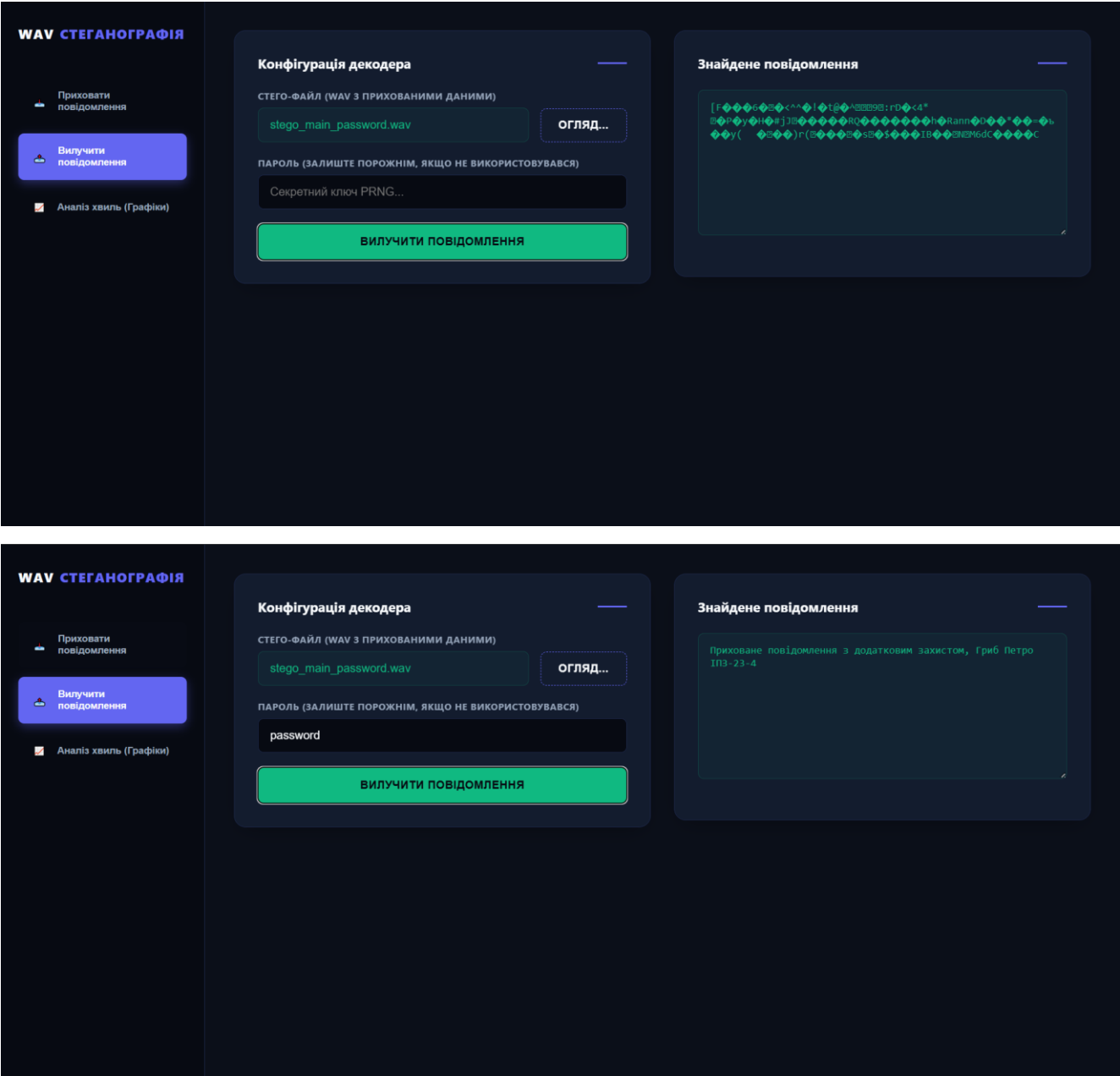


Рис. 2.2-2.3. Спроба вилучити приховане повідомлення із та без паролю

Посилання на GitHub: <https://github.com/ipz234-gpyu/Steganography-Lab4>

Висновки: дослідити основні принципи аудіостеганографії; реалізувати алгоритм LSB для приховування текстових даних у WAV-файлі, а також функціонал для вилучення прихованого повідомлення з аудіофайлу.